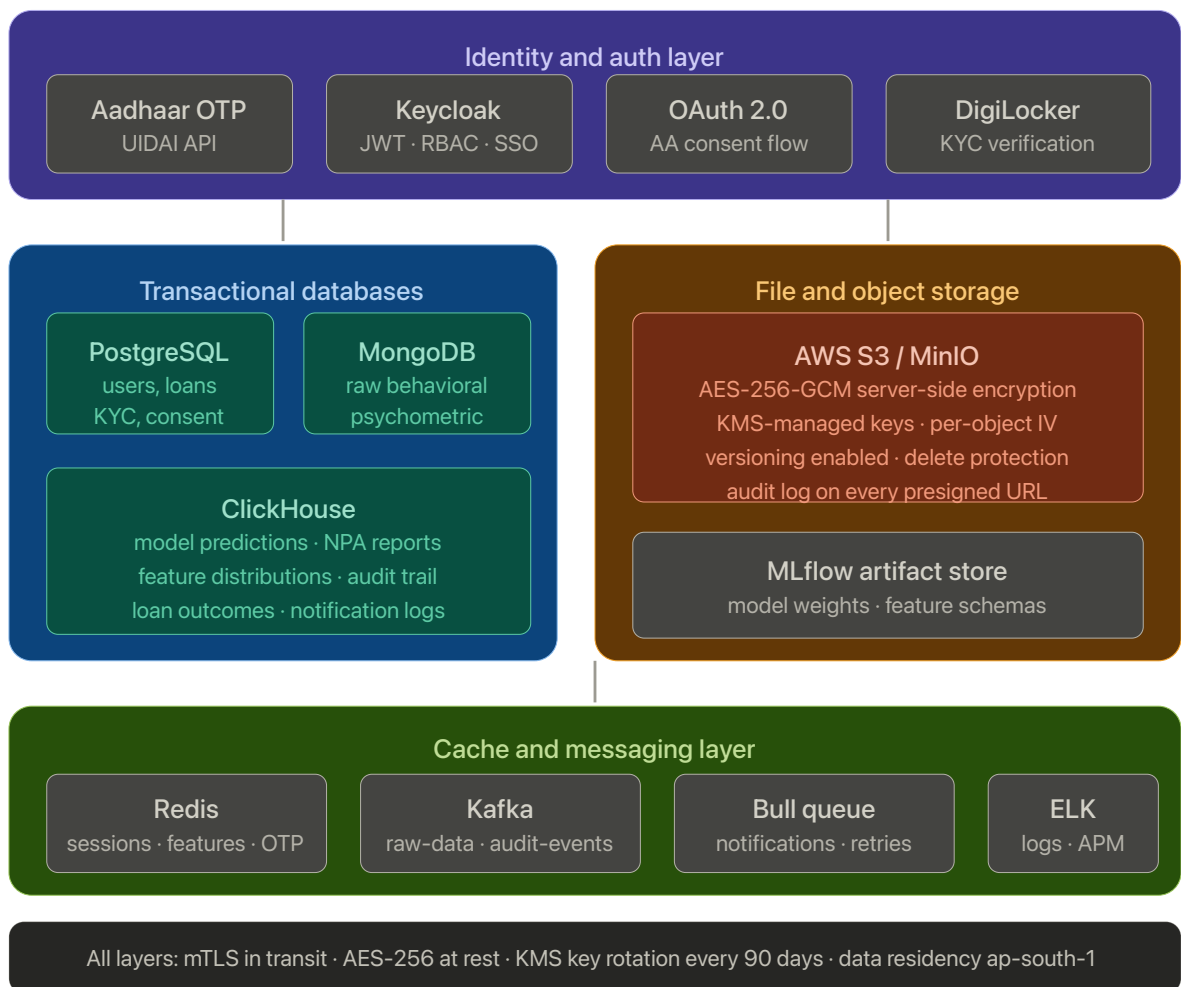
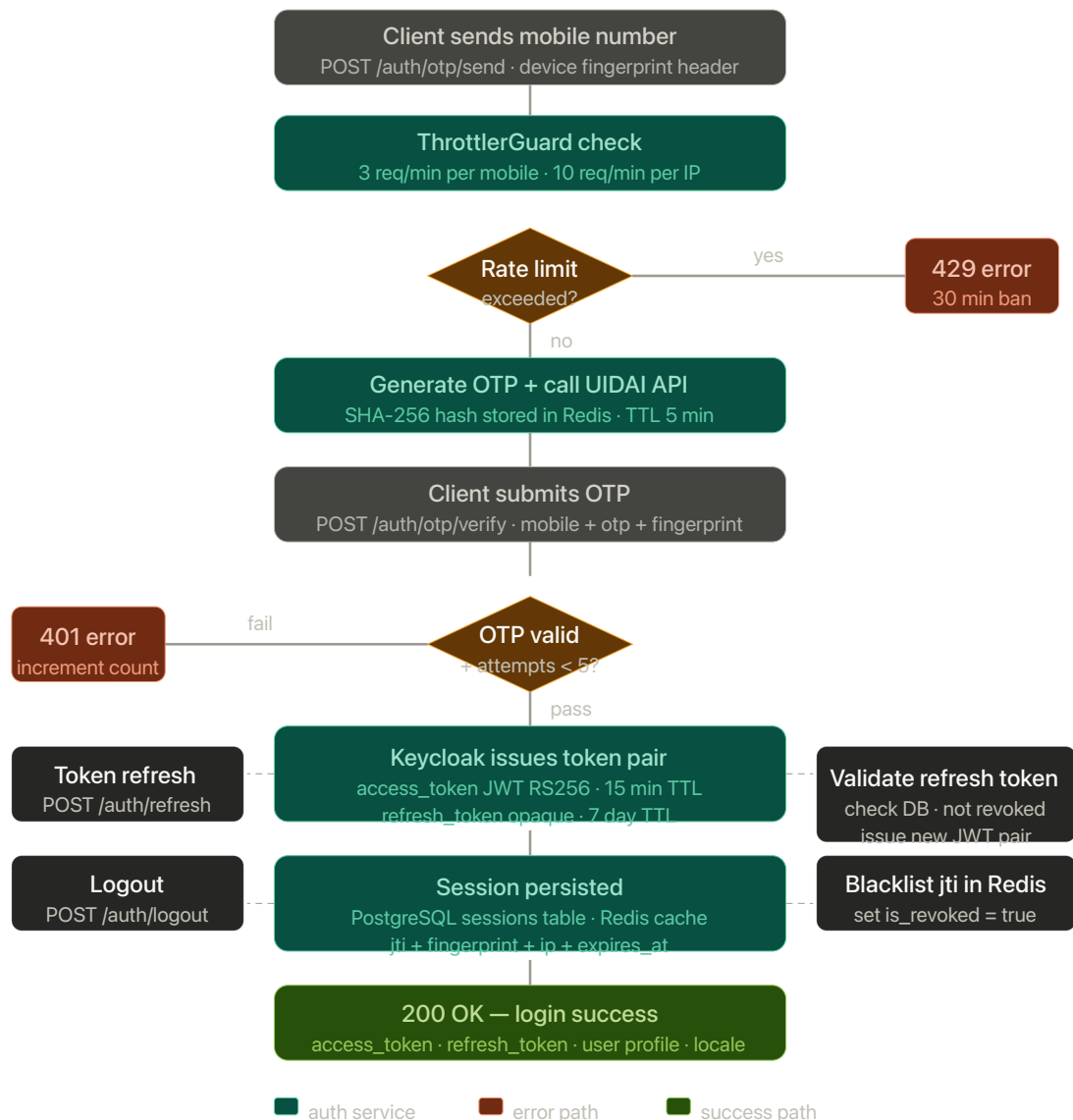




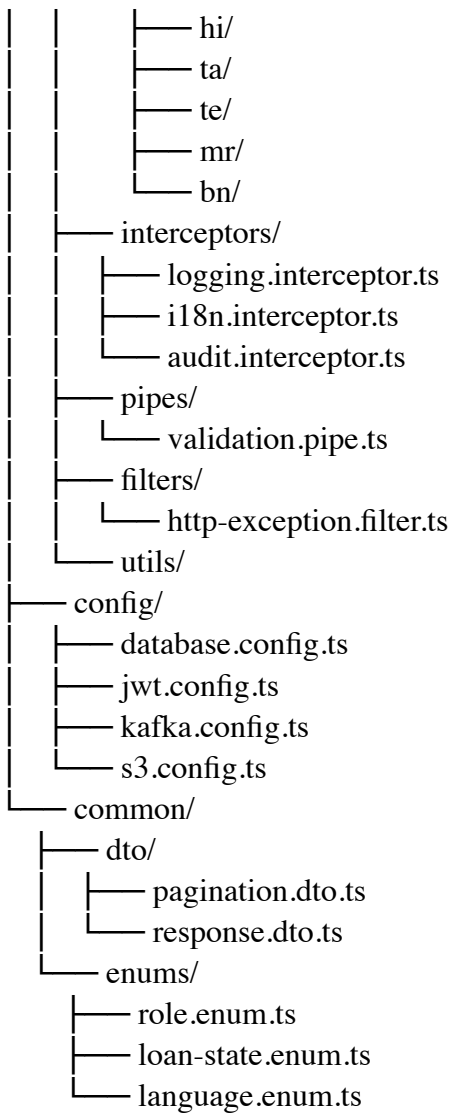
The login flow is the most security-critical path in the system — here it is step by step.



NestJS backend — complete plan

Project structure

```
src/
├── main.ts
├── app.module.ts
├── modules/
│   ├── auth/
│   │   ├── auth.module.ts
│   │   ├── auth.controller.ts
│   │   ├── auth.service.ts
│   │   ├── auth.repository.ts
│   │   ├── auth.dto.ts
│   │   ├── auth.types.ts
│   │   ├── auth.spec.ts
│   │   ├── strategies/
│   │   │   ├── jwt.strategy.ts
│   │   │   └── refresh.strategy.ts
│   │   ├── guards/
│   │   │   ├── jwt-auth.guard.ts
│   │   │   └── roles.guard.ts
│   │   └── decorators/
│   │       ├── roles.decorator.ts
│   │       └── current-user.decorator.ts
│   ├── user/
│   ├── kyc/
│   ├── consent/
│   ├── document/
│   ├── scoring/
│   ├── loan/
│   ├── emi/
│   ├── notification/
│   ├── cms/
│   ├── audit/
│   └── admin/
├── shared/
│   ├── database/
│   ├── cache/
│   ├── kafka/
│   ├── storage/
│   ├── i18n/
│   │   ├── i18n.module.ts
│   │   └── translations/
│   │       └── en/
```



main.ts bootstrap

typescript

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';
import { ValidationPipe, VersioningType } from '@nestjs/common';
import { SwaggerModule, DocumentBuilder } from '@nestjs/swagger';
import { I18nValidationExceptionFilter, I18nValidationPipe } from 'nestjs-i18n';
import helmet from 'helmet';
import * as compression from 'compression';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  app.use(helmet());
  app.use(compression());

  app.enableCors({ origin: process.env.ALLOWED_ORIGINS?.split(','), credentials: true });
```

```

app.enableVersioning({ type: VersioningType.URI, defaultVersion: '1' });

app.useGlobalPipes(
  new I18nValidationPipe(),
  new ValidationPipe({ whitelist: true, forbidNonWhitelisted: true, transform: true }),
);

app.useGlobalFilters(new I18nValidationExceptionFilter());

const swaggerConfig = new DocumentBuilder()
  .setTitle('CredSaathi API')
  .setDescription('AI credit scoring platform API')
  .setVersion('1.0')
  .addBearerAuth()
  .build();
const doc = SwaggerModule.createDocument(app, swaggerConfig);
SwaggerModule.setup('api/docs', app, doc);

await app.listen(3000);
}
bootstrap();

```

Full REST endpoint documentation

All endpoints prefixed `/api/v1/`. All responses wrapped in `{ success, data, error, request_id, timestamp }`.

Auth module — `/auth`

POST `/auth/otp/send` Request: `{ mobile: string }` + X-Device-Fingerprint header Response: `{ message: string, expires_in: 300 }` Guards: ThrottlerGuard (3/min per mobile, 10/min per IP) Description: Hashes OTP with SHA-256, stores in Redis with TTL 5 min, calls UIDAI SMS gateway.

POST `/auth/otp/verify` Request: `{ mobile: string, otp: string, device_fingerprint: string, fcm_token?: string }` Response: `{ access_token, refresh_token, user: { id, role, name, locale }, is_new_user: boolean }` Guards: ThrottlerGuard Description: Verifies OTP hash match, checks attempt count (<5), issues JWT pair from Keycloak, writes session to PostgreSQL and Redis.

POST `/auth/refresh` Request: `{ refresh_token: string }` (Bearer header) Response: `{ access_token, refresh_token }` Guards: RefreshTokenGuard Description: Validates opaque refresh token against PostgreSQL sessions, checks `is_revoked`, issues new token pair, rotates refresh token.

POST `/auth/logout` Request: (Bearer `access_token` header) Response: `{ message: 'logged_out' }` Guards: JwtAuthGuard Description: Extracts `jti` from JWT, blacklists in Redis with remaining TTL, sets `sessions.is_revoked = true`.

POST /auth/logout/all Request: (Bearer header) Response:
{ sessions_revoked: number } Guards: JwtAuthGuard Description: Revokes all active sessions for the user (all devices).

GET /auth/me Response: full user profile + active consents + latest score band Guards: JwtAuthGuard

User module — /users

GET /users/profile Response: { id, name, mobile, role, kyc_status, onboarding_step, locale, created_at } Guards: JwtAuthGuard

PATCH /users/profile Request: { name?: string, email?: string, locale?: string } Response: updated profile Guards: JwtAuthGuard

GET /users/:id (lender/admin) Guards: JwtAuthGuard + RolesGuard(lender, admin)

DELETE /users/me Description: DPDP Act deletion request — soft deletes user, triggers 72-hour data purge workflow Guards: JwtAuthGuard

POST /users/onboarding/step Request: { step: OnboardingStep, data: Record<string, any> } Description: Tracks multi-step onboarding progress (personal info → KYC → consent → questionnaire → score) Guards: JwtAuthGuard

KYC module — /kyc

POST /kyc/initiate Request: { aadhaar_last4: string } + Aadhaar OTP re-verification Response: { kyc_session_id, digilocker_auth_url } Guards: JwtAuthGuard

POST /kyc/complete Request: { kyc_session_id, digilocker_code } (OAuth callback) Response: { status: 'verified', fields: ['name', 'dob', 'address'] } Description: Exchanges DigiLocker OAuth code, pulls PAN + Aadhaar data, runs liveness FaceMatch check, writes kyc_records row.

GET /kyc/status Response: { status, verified_at, documents_uploaded } Guards: JwtAuthGuard

POST /kyc/liveness Request: multipart selfie image Response: { match_score: float, passed: boolean } Description: FaceMatch SDK call against Aadhaar photo.

Consent module — /consent

GET /consent Response: { sources: [{ source, is_active, granted_at, scope }] } Guards: JwtAuthGuard

POST /consent/grant Request: { source: DataSource, scope: 'read' | 'read_aggregate', aa_handle: string } Response: { consent_id, consent_hash, granted_at } Guards: JwtAuthGuard Description: Writes immutable consent record, triggers AA API authorisation flow.

DELETE /consent/:source Response: { revoked_at, data_deletion_scheduled_at } Guards: JwtAuthGuard Description: Marks consent inactive, schedules data deletion, notifies AA to cancel FIP access.

GET /consent/audit Response: full consent history (grant + revoke events) for the user
Guards: JwtAuthGuard

Document module — /documents

POST /documents/upload Request: multipart form — file (PDF/JPEG/PNG), doc_type (enum) Response: { document_id, doc_type, uploaded_at } Guards: JwtAuthGuard Description: Validates file type and size (<10 MB), computes SHA-256 hash, encrypts with AES-256-GCM, streams to S3, writes documents row.

GET /documents Response: list of uploaded documents (no file content, only metadata)
Guards: JwtAuthGuard

GET /documents/:id/url Response: { presigned_url, expires_in: 900 } Guards: JwtAuthGuard Description: Generates 15-minute presigned S3 URL, logs access in audit trail.

DELETE /documents/:id Response: { scheduled_deletion_at } Guards: JwtAuthGuard Description: Soft delete in PostgreSQL; hard S3 delete after 72 hours.

Scoring module — /score

POST /score/generate Request: { loan_amount_requested?: number } Response: { score, risk_band, shap_factors: [{ feature, contribution, direction }], generated_at, model_version } Guards: JwtAuthGuard Description: Fetches feature vector from Redis (or recomputes from MongoDB), calls BentoML scoring service, persists result, returns score with SHAP breakdown. P95 latency target: <3 sec.

GET /score/latest Response: latest score record including full SHAP report Guards: JwtAuthGuard

GET /score/history Response: paginated list of all score events Guards: JwtAuthGuard

GET /score/:user_id (lender only) Guards: JwtAuthGuard + RolesGuard(lender, admin)
Description: Lender view — full score + SHAP waterfall for a specific applicant.

GET /score/:user_id/explain Response: human-readable i18n explanation of each SHAP factor in borrower's language Guards: JwtAuthGuard + RolesGuard(lender, admin)

Loan module — /loans

POST /loans Request: { amount: number, tenure_months: number, purpose: string, lender_id?: uuid } Response: { loan_id, state: 'submitted', score_id, applied_at } Guards: JwtAuthGuard + RolesGuard(borrower)

GET /loans Response: paginated loan list with current state Guards: JwtAuthGuard

GET /loans/:id Response: full loan detail + state history + EMI schedule Guards: JwtAuthGuard

PATCH /loans/:id/decision Request: { decision: 'approved' | 'rejected', interest_rate?: number, reason?: string } Response: updated loan with new state Guards: JwtAuthGuard + RolesGuard(lender) Description: State machine transition with row lock. If amount > ₹2,00,000 requires second lender approval flag.

POST /loans/:id/disburse Guards: JwtAuthGuard + RolesGuard(lender) Description: Triggers IMPS/NEFT payment via payment gateway, updates loan state to **disbursed**, generates EMI schedule.

GET /loans/pipeline (lender) Query params: ? state=submitted&risk_band=low&page=1&limit=20 Guards: JwtAuthGuard + RolesGuard(lender)

EMI module — /emi

GET /emi/schedule/:loan_id Response: full EMI schedule with due dates, amounts, status Guards: JwtAuthGuard

POST /emi/repayment Request: { loan_id, emi_id, amount_paid, utr_number, payment_mode } Response: { emi_status, days_late, loan_state } Guards: JwtAuthGuard Description: Bank webhook also hits this endpoint. Computes DPD, updates EMI status, checks 90-DPD for default trigger.

GET /emi/upcoming Response: next 3 EMIs due for the authenticated borrower Guards: JwtAuthGuard

CMS module — /cms

GET /cms/content/:key Query: ?locale=hi Response: { key, content, locale, version } Guards: public (no auth for read)

GET /cms/faqs Query: ?locale=ta&category=loan Response: paginated FAQ list in requested locale

GET /cms/loan-products Response: available loan products with rates, tenure, eligibility criteria

GET /cms/questionnaire Query: ?locale=te Response: psychometric questionnaire in requested language

POST /cms/content (admin only) PATCH /cms/content/:key (admin only)

DELETE /cms/content/:key (admin only) Guards: JwtAuthGuard + RolesGuard(admin)

Admin module — /admin

GET /admin/model/metrics Response: current AUC, KS, PSI, fairness metrics across all groups Guards: JwtAuthGuard + RolesGuard(admin)

POST /admin/model/retrain Body: { force: boolean } Guards: JwtAuthGuard + RolesGuard(admin) Description: Triggers Airflow DAG via HTTP.

GET /admin/reports/compliance Query: ?from=2024-01&to=2024-03 Guards: JwtAuthGuard + RolesGuard(admin, compliance)

GET /admin/audit/:entity_type/:entity_id Guards: JwtAuthGuard + RolesGuard(admin, compliance)

i18n — complete implementation

Install: `npm install nestjs-i18n`

`i18n.module.ts`:

typescript

```
import { I18nModule, AcceptLanguageResolver, QueryResolver, HeaderResolver } from 'nestjs-i18n';
```

```
import * as path from 'path';
```

```
I18nModule.forRoot({
  fallbackLanguage: 'en',
  loaderOptions: {
    path: path.join(__dirname, '/translations/'),
    watch: true,
  },
  resolvers: [
    { use: QueryResolver, options: ['lang', 'locale'] },
    { use: HeaderResolver, options: ['x-locale'] },
    AcceptLanguageResolver,
  ],
})
```

Supported languages: **en** (English), **hi** (Hindi), **ta** (Tamil), **te** (Telugu), **mr** (Marathi), **bn** (Bengali).

Translation file structure — **src/shared/i18n/translations/hi/auth.json**:

```
json
{
  "otp": {
    "sent": "OTP आपके मोबाइल {mobile} पर भेजा गया है",
    "verified": "OTP सत्यापित हुआ। आपका स्वागत है, {name}!",
    "invalid": "गलत OTP। {attempts} प्रयास शेष हैं",
    "expired": "OTP समाप्त हो गया। कृपया पुनः प्रयास करें",
    "blocked": "बहुत अधिक प्रयास। 30 मिनट बाद पुनः प्रयास करें"
  },
  "errors": {
    "unauthorized": "आप अधिकृत नहीं हैं",
    "session_expired": "सत्र समाप्त हो गया। पुनः लॉगिन करें"
  }
}
```

Translation file structure — **src/shared/i18n/translations/hi/score.json**:

```
json
{
  "bands": {
    "very_low_risk": "बहुत कम जोखिम",
    "low_risk": "कम जोखिम",
    "medium_risk": "मध्यम जोखिम",
    "high_risk": "उच्च जोखिम",
    "very_high_risk": "बहुत उच्च जोखिम"
  },
  "factors": {
    "bill_ontime_ratio": "समय पर बिल भुगतान",
    "cash_flow_stability": "नकद प्रवाह स्थिरता",
    "spending_volatility": "खर्च की अनिश्चितता",
    "geo_stability": "स्थान स्थिरता"
  },
  "explanation": "आपका {factor} आपके स्कोर में {direction} {points} अंक योगदान देता है"
}
```

Translation files exist in all 6 languages for these namespaces: **auth**, **user**, **kyc**, **consent**, **score**, **loan**, **emi**, **errors**, **notifications**, **cms**.

Using i18n in a service:

```
typescript
import { I18nService, I18nContext } from 'nestjs-i18n';
```

```
@Injectable()
export class AuthService {
  constructor(private readonly i18n: I18nService) {}

  async sendOtp(mobile: string, lang: string) {
    const message = await this.i18n.translate('auth.otp.sent', {
      lang,
      args: { mobile: mobile.slice(-4).padStart(10, '*') },
    });
    return { message };
  }
}
```

Using i18n in a controller (auto-detects from **Accept-Language** header):

```
typescript
@Post('otp/send')
async sendOtp(@Body() dto: SendOtpDto, @I18n() i18n: I18nContext) {
  return this.authService.sendOtp(dto.mobile, i18n.lang);
}
```

i18n error messages in validation DTOs:

```
typescript
import { IsNotEmpty, IsPhoneNumber } from 'class-validator';
import { i18nValidationMessage } from 'nestjs-i18n';

export class SendOtpDto {
  @IsNotEmpty({ message: i18nValidationMessage('validation.mobile.required') })
  @IsPhoneNumber('IN', { message: i18nValidationMessage('validation.mobile.invalid') })
  mobile: string;
}
```

Language preference stored on the user record (**locale** field). Resolved in priority order:

explicit **?lang=** query param → **X-Locale** header → **Accept-Language** header → user's stored **locale** → fallback **en**.

CMS module — full design

The CMS manages all dynamic text content so non-developers can update FAQs, loan product descriptions, questionnaire wording, notification templates, and onboarding copy without a code deploy.

cms_content PostgreSQL table:

```

sql
CREATE TABLE cms_content (
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  key     VARCHAR(200) NOT NULL,
  locale  VARCHAR(10) NOT NULL,
  namespace VARCHAR(50) NOT NULL,
  content JSONB NOT NULL,
  version INTEGER DEFAULT 1,
  is_active BOOLEAN DEFAULT true,
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(key, locale)
);
CREATE INDEX idx_cms_key_locale ON cms_content(key, locale, is_active);
cms_faqs table:

```

```

sql
CREATE TABLE cms_faqs (
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  category VARCHAR(50),
  question TEXT NOT NULL,
  answer  TEXT NOT NULL,
  locale  VARCHAR(10) NOT NULL,
  sort_order INTEGER DEFAULT 0,
  is_active BOOLEAN DEFAULT true
);
cms_loan_products table:

```

```

sql
CREATE TABLE cms_loan_products (
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name     VARCHAR(200),
  min_amount INTEGER,
  max_amount INTEGER,
  min_tenure_months INTEGER,
  max_tenure_months INTEGER,
  interest_rate_min DECIMAL(5,2),
  interest_rate_max DECIMAL(5,2),
  eligibility_json JSONB,
  description JSONB,
  is_active BOOLEAN DEFAULT true
);
cms_questionnaire table — stores versioned psychometric questionnaire with all language variants:

```

```
sql
CREATE TABLE cms_questionnaire_questions (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  version     VARCHAR(10) NOT NULL,
  q_number    INTEGER NOT NULL,
  group_name  VARCHAR(50),
  question    JSONB NOT NULL,
  options     JSONB NOT NULL,
  scoring_rule JSONB NOT NULL,
  is_active   BOOLEAN DEFAULT true
);
```

Where `question` JSONB looks like:

```
json
{
  "en": "If you won ₹10,000 today, what would you do?",
  "hi": "यदि आपको आज ₹10,000 मिलें, तो आप क्या करेंगे?",
  "ta": "இன்று ₹10,000 கிடைத்தால் என்ன செய்வீர்கள்?",
  "te": "మీకు నేడు ₹10,000 వస్తే ఏం చేస్తారు?",
  "mr": "आज तुम्हाला ₹10,000 मिळाले तर तुम्ही काय कराल?",
  "bn": "আজ ₹10,000 পেলে আপনি কী করবেন?"
}
```

CMS service reads with Redis caching — cache key `cms:{key}:{locale}`, TTL 1 hour.

Admin `PATCH` flushes the relevant cache key. This means CMS changes go live within 1 hour max without a deploy.

Login documentation — complete reference

Endpoint 1 — **POST** `/api/v1/auth/otp/send`

Headers:

Content-Type: `application/json`

X-Device-Fingerprint: {string} (SHA-256 hash of device identifiers)

Body:

```
{ "mobile": "+919876543210" }
```

Responses:

200: { "success": true, "data": { "message": "OTP sent to +91xxxxxx3210", "expires_in": 300 } }

422: { "success": false, "error": { "code": "VALIDATION_ERROR", "fields": { "mobile": "Invalid Indian mobile number" } } }

429: { "success": false, "error": { "code": "RATE_LIMIT_EXCEEDED", "retry_after": 1800 } }

Endpoint 2 — **POST** `/api/v1/auth/otp/verify`

Headers:

Content-Type: application/json
X-Device-Fingerprint: {string}
Accept-Language: hi (triggers Hindi i18n on response messages)

Body:

```
{
  "mobile": "+919876543210",
  "otp": "482931",
  "fcm_token": "device_push_token_optional"
}
```

Success response (200):

```
{
  "success": true,
  "data": {
    "access_token": "eyJhbGciOiJSUzI1NiJ9...",
    "refresh_token": "8f3c7a1b9d4e2f6...",
    "token_type": "Bearer",
    "expires_in": 900,
    "user": {
      "id": "uuid",
      "name": "राहुल शर्मा",
      "mobile": "+91xxxxxx3210",
      "role": "borrower",
      "locale": "hi",
      "kyc_status": "pending",
      "onboarding_step": "kyc",
      "score_band": null
    },
    "is_new_user": true
  },
  "request_id": "req_01J...",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

Error responses:

401: { "error": { "code": "OTP_INVALID", "message": "गलत OTP। 3 प्रयास शेष हैं",
"attempts_remaining": 3 } }
401: { "error": { "code": "OTP_EXPIRED", "message": "OTP समाप्त हो गया" } }
423: { "error": { "code": "ACCOUNT_LOCKED", "message": "बहुत अधिक प्रयास। 30 मिनट बाद
प्रयास करें", "locked_until": "2024-01-15T11:00:00Z" } }

Endpoint 3 – POST /api/v1/auth/refresh

Headers:

Authorization: Bearer {refresh_token}

Success response (200):

```
{
  "access_token": "eyJhbGciOiJSUzI1NiJ9...",
  "refresh_token": "new_opaque_token...",
  "expires_in": 900
}
```

Error responses:

```
401: { "error": { "code": "REFRESH_TOKEN_INVALID" } }
401: { "error": { "code": "REFRESH_TOKEN_EXPIRED" } }
401: { "error": { "code": "SESSION_REVOKED" } }
```

Endpoint 4 — POST /api/v1/auth/logout

Headers:

Authorization: Bearer {access_token}

Success response (200):

```
{ "success": true, "data": { "message": "Logged out successfully" } }
```

Behaviour:

- Extracts jti from JWT payload
- Sets Redis key blacklist:{jti} = 1 with TTL = remaining token lifetime
- Sets PostgreSQL sessions.is_revoked = true WHERE jwt_jti = {jti}
- All subsequent requests with the same access_token return 401 TOKEN_BLACKLISTED

JWT payload structure

json

```
{
  "sub": "user-uuid",
  "jti": "unique-token-id",
  "role": "borrower",
  "locale": "hi",
  "iat": 1705312200,
  "exp": 1705313100,
  "iss": "credsathi-auth",
  "aud": "credsathi-api"
}
```

Per-request auth guard pipeline

Every protected request passes through this chain in NestJS before reaching the controller:

1. **JwtAuthGuard** — extracts Bearer token, verifies RS256 signature using Keycloak public key, checks **exp**
2. **Redis blacklist check** — **EXISTS blacklist:{jti}** — if exists, 401 immediately
3. **RolesGuard** — reads **@Roles()** decorator on handler, checks JWT **role** claim

4. `I18nInterceptor` — reads locale from JWT claims or `Accept-Language` header, sets `I18nContext` for the request
5. `LoggingInterceptor` — logs `request_id`, method, path, `user_id`, duration
6. `AuditInterceptor` — emits audit event to Kafka for write operations (POST, PATCH, DELETE)